Université Paris Dauphine – PSL

Large Language Models

LORA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Group: IBIF

Students:

Bochra Lafifi bochra.lafifi@dauphine.eu
Fatima Al Harake fatima.al-harake@dauphine.eu
Ibtissam Fatma Chouchou ibtissam.chouchou@dauphine.eu
Ines Manel Boumazouza ines.boumazouza@dauphine.eu

December 14, 2025

Contents

1 Introduction **2**

2 LoRA: Low-Rank Adaptation **2**
2.1 The Mathematics Behind LoRA 2

3 Where LoRA is Applied in Transformers **3**
3.1 Target Weight Matrices 3
3.2 Empirical Findings on Weight Selection 3

4 Experimental Results **3**
4.1 Performance Comparison on GPT-3 (175B) 3
4.2 Rank Study 4

5 Proposed Experiment & Implementation **4**
5.1 Implementation Details 4
5.2 Results and Discussion 5

6 Conclusion **6**

Appendix: AI Usage **8**

1 Introduction

Modern language models have grown enormously, often reaching hundreds of billions of parameters. While full fine-tuning can adapt these models to specific tasks, it quickly becomes impractical: each task requires storing a full model copy, training demands massive memory for optimizer states, and deployment is cumbersome without specialized hardware.

2 LoRA: Low-Rank Adaptation

LoRA [1] addresses these challenges by freezing the pre-trained weights and learning only small, trainable low-rank matrices injected into each layer. This approach drastically reduces the number of trainable parameters (up to 10,000× fewer) while preserving or even improving performance, enabling efficient adaptation of massive models on standard hardware. By focusing on a low-dimensional subspace of task-specific knowledge, LoRA combines both memory efficiency and ease of deployment without altering the model's forward pass.

2.1 The Mathematics Behind LoRA

Core Concept: Low-Rank Decomposition

For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d\times k}$, instead of learning the full update ΔW, LoRA represents it as

$$\Delta W = BA,$$

where $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$, and $r \ll \min(d, k)$ is the rank constraint. The forward pass then becomes

$$h = W_0\, x + \Delta W\, x = W_0\, x + BA\, x \tag{1}$$

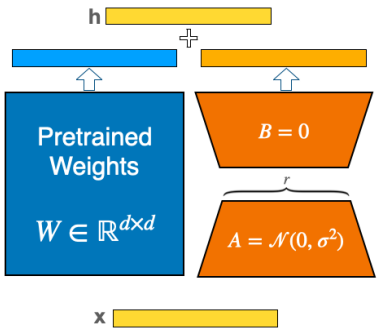

Figure 1: The reparametrization from [1], only A and B are trained.

The Intrinsic Rank Hypothesis

Research shows that learned model updates during adaptation reside in a low "intrinsic dimension" [2].
Empirical Evidence from the Paper:

- For GPT-3 175B, $r = 1$ or 2 suffices for many tasks
- Even when full rank $d = 12{,}288$, rank-1 updates achieve 91%+ accuracy
- This suggests task-specific adaptations require very few degrees of freedom

Initialization Strategy

Critical Design Choice: $A \sim \mathcal{N}(0, \sigma^2)$ (Gaussian initialization), $B = 0$ (zero initialization).
Result: $\Delta W = BA = 0$ at training start, so the model begins from the pre-trained state.
Scaling factor: $\Delta W x$ is scaled by $\frac{\alpha}{r}$ where α is a constant (typically set to r for simplicity).

Parameter Count Analysis

For a Transformer with:

- $d_{\text{model}} = 12{,}288$ (GPT-3 175B)
- 96 layers
- Applying LoRA to W_q and W_v with $r = 4$

Trainable parameters:

$$|\Theta| = 2 \times L \times d_{\text{model}} \times r = 2 \times 96 \times 12{,}288 \times 4 \approx 9.4\text{M parameters} \tag{2}$$

Reduction factor: 18,617× (vs. 175B!)

3 Where LoRA is Applied in Transformers

3.1 Target Weight Matrices

In Transformers, LoRA is mainly applied to linear projections in the self-attention and MLP blocks. In the self-attention module, this includes the query, key, value, and output projections (W_q, W_k, W_v, W_o), while in the MLP it can be applied to the feed-forward layers. In practice, attention projections dominate performance gains.

3.2 Empirical Findings on Weight Selection

Empirical results show that adapting a subset of attention matrices is sufficient. Table 1 reports results with a fixed budget of 18M trainable parameters.

Table 1: Effect of adapting different weight matrices (18M trainable parameters)

Configuration	Rank	WikiSQL	MNLI
W_q only	8	70.4	91.0
W_k only	8	70.0	90.8
W_v only	8	73.0	91.0
$W_q + W_v$	**4**	**73.7**	**91.3**
All attention	2	73.7	91.7

Adapting both W_q and W_v with a moderate rank consistently outperforms single-matrix adaptations, offering a strong performance–efficiency trade-off.

4 Experimental Results

4.1 Performance Comparison on GPT-3 (175B)

Table 2 compares LoRA with full fine-tuning and other parameter-efficient methods on three benchmarks. Despite training only 4.7M parameters, LoRA matches or outperforms full fine-tuning and adapters.

LoRA achieves comparable or superior performance with over 37,000× fewer trainable parameters than full fine-tuning.

Table 2: Performance comparison on GPT-3 175B

Method	Params	WikiSQL	MNLI-m	SAMSum
Full FT	175B	73.8	89.5	52.0/28.0/44.5
Adapter$_H$	40M	73.2	91.5	53.2/29.0/45.1
Prefix	20M	70.1	89.5	50.8/27.3/43.5
LoRA	**4.7M**	**73.4**	**91.7**	**53.8/29.8/45.9**

4.2 Rank Study

A rank ablation reveals that performance is largely insensitive to rank. For $W_q + W_v$ on GPT-3, $r = 1$ already achieves 73.4% on WikiSQL and 91.3% on MNLI, while higher ranks up to $r = 64$ offer negligible gains. This confirms that most task-specific adaptation lies in just one or two principal directions.

5 Proposed Experiment & Implementation

Since the LoRA paper proposes an efficient fine-tuning approach, we conducted a case study applying this technique to a practical task and comparing it with other parameter-efficient fine-tuning methods. Our experimental setup consists of: **BERT** [3] as the pre-trained model, fine-tuned for **emotion detection** (6-class sequence classification) using the `dair-ai/emotion`[1] HuggingFace dataset. We implemented as a baseline for comparison a full fine-tuned model. Comparisons were performed during training (loss, accuracy, number of trainable parameters) and inference (test accuracy, F1-score, inference latency). Our models were trained using HuggingFace Trainer API with evaluation at each epoch. Training was conducted on Google Colab with T4 GPU.

5.1 Implementation Details

We loaded the `emotion` dataset and selected 3,000 samples for training, 500 for validation, and 500 for testing (shuffled with seed 42). Text samples were tokenized using BERT's uncased tokenizer with truncation and padding to maximum length 128 tokens, then converted to PyTorch tensors for training.

BERT with LoRA (implementation from scratch):

We implemented LoRA from scratch following the paper's mathematical formulation. Our implementation consists of two classes.

- The `LoRALayer` class wraps any given layer with low-rank decomposition by initializing matrices $A \in \mathbb{R}^{d\times r}$ using Kaiming uniform initialization and $B \in \mathbb{R}^{r\times d}$ with zeros (ensuring initial $\Delta W = 0$), and then computing the forward pass as expressed in 1.
- The `BertLoRAForSequenceClassification` class loads pre-trained BERT-base-uncased, injects LoRA layers into the attention projection matrices (Query, Key, Value) across all 12 transformer layers, adds a classification head for the 6-class task, and freezes all original BERT parameters while keeping only LoRA matrices and the classification head trainable.

To find optimal hyperparameters, we performed a grid search over rank values $r \in \{2, 8, 16\}$, epochs $\in \{5, 10\}$, and batch sizes $\in \{16, 32\}$ (24 combinations total). The best configuration was

[1] https://huggingface.co/datasets/dair-ai/emotion

selected based on validation accuracy. All experiments used scaling factor $\alpha = 16$, learning rate 3×10^{-4}, and weight decay 0.01.

Full Fine-tuning:

We implemented standard BERT fine-tuning where all 109.5M parameters are trainable, using a custom `BertForSequenceClassification` class that adds a dropout layer and classification head on top of pre-trained BERT.

5.2 Results and Discussion

Having implemented both LoRA and full fine-tuning as described above, we now present the comparative analysis across the two evaluation dimensions mentioned earlier: training and inference. We begin by reporting the optimal hyperparameters identified through grid search for LoRA. We then analyze training performance through loss curves, accuracy evolution, and efficiency metrics (trainable parameters and training time). Next, we evaluate inference performance on the test set in terms of accuracy and latency. Finally, we provide a general discussion interpreting these results in light of the LoRA paper's claims.

Hyperparameter Tuning

The grid search over 12 combinations identified the optimal configuration as $r = 16$, 10 epochs, and batch size 16, achieving a validation accuracy of 87.8%. This configuration was used for all subsequent comparisons.

Training Performance Analysis

Figure 2 presents the training loss, validation loss, and validation accuracy curves for both methods over 10 epochs.

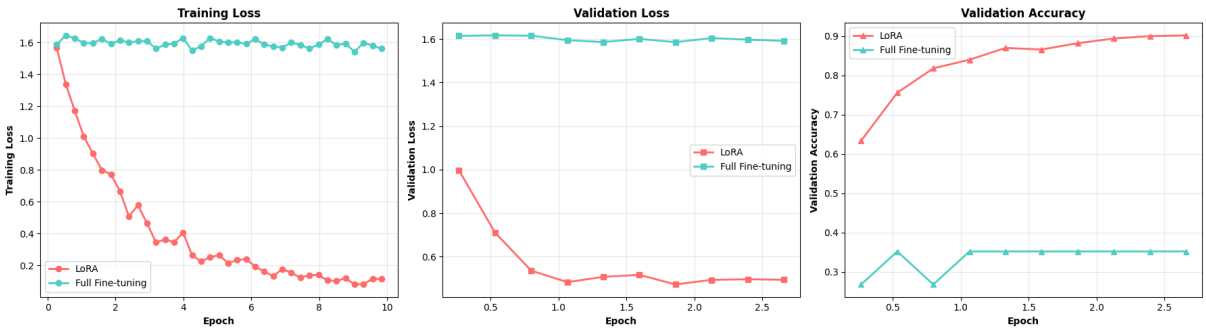

Figure 2: Training and validation curves comparison: (left) training loss, (center) validation loss, (right) validation accuracy.

The training curves show clear differences between the two methods. LoRA demonstrates smooth learning: training loss decreases steadily from 1.6 to 0.1, while validation accuracy improves consistently from 62% to 90.2%. Full fine-tuning, however, fails to learn effectively: training loss oscillates around 1.6 without improvement, and validation accuracy remains stuck at 35.2%. This poor performance occurs because training 109M parameters requires both more epochs and larger datasets than our setup provides (10 epochs, 3,000 samples). LoRA's low-rank constraints enable effective learning even with limited data and computational resources.
Table 3 summarizes the training efficiency metrics.

Table 3: Training efficiency comparison

Method	Trainable Params	% of Full	Training Time (s)	Val Acc
Full Fine-tuning	109,486,854	100%	374.1	0.352
LoRA ($r = 16$)	889,350	0.81%	218.7	0.902

LoRA achieves an impressive reduction in trainable parameters (from 109.5M to 889K), representing only 0.81% of full fine-tuning's parameters. Additionally, training time is reduced by 35.5% (from 374.1s to 218.7s), demonstrating improved efficiency. The dramatic performance gap (90.2% vs 35.2% validation accuracy) highlights the benefit of tuning following LoRA.

Inference Performance Analysis

Table 4 presents the test set performance and inference latency.

Table 4: Inference performance on test set (500 samples)

Method	Test Accuracy	Avg Latency (s/sample)
Full Fine-tuning	0.386	0.0113
LoRA ($r = 16$)	0.890	0.0174

LoRA achieves 89.0% test accuracy compared to 38.6% for full fine-tuning, confirming the validation results and demonstrating strong generalization. The inference latency shows no overhead, as expected since LoRA matrices can be merged with frozen weights at deployment.

General Discussion

Our results provide strong empirical support for the LoRA paper's core hypotheses:
Low-rank hypothesis validation: The fact that LoRA with rank $r = 16$ achieves 89% test accuracy while training only 0.81% of parameters demonstrates that task-specific adaptations indeed lie in a low-dimensional subspace. The important parameter reduction with superior performance validates this fundamental claim.
Practical advantages: Beyond parameter efficiency, in general LoRA achieves faster training and enables efficient multi-task deployment by sharing the frozen base model across tasks. The minimal inference overhead makes LoRA deployment-ready.
Limitations: Our study has several constraints. First, the small dataset (3,000 samples) may exaggerate LoRA's regularization advantage; larger datasets might narrow the performance gap with full fine-tuning. Second, we evaluated only a single classification task; generation tasks or other NLP problems may show different trade-offs. Third, the poor full fine-tuning performance suggests our setup may not be optimal for that baseline. Finally, we explored only attention projections (Q, K, V); applying LoRA to feed-forward layers might yield additional benefits.

6 Conclusion

LoRA offers a powerful solution for adapting large language models by drastically reducing trainable parameters while maintaining competitive performance. Our BERT implementation confirms the paper's findings: LoRA achieves results comparable to full fine-tuning with significantly lower memory requirements, faster training, and zero inference latency. These advantages make it particularly valuable for deploying multiple task-specific models efficiently. Beyond NLP, LoRA extends to computer vision and speech processing, demonstrating versatility across domains [4].

References

[1] Edward J Hu et al. "Lora: Low-rank adaptation of large language models." In: *ICLR* 1.2 (2022), p. 3.

[2] Armen Aghajanyan, Sonal Gupta, and Luke Zettlemoyer. "Intrinsic dimensionality explains the effectiveness of language model fine-tuning". In: *Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing (volume 1: long papers)*. 2021, pp. 7319–7328.

[3] Jacob Devlin et al. "Bert: Pre-training of deep bidirectional transformers for language understanding". In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019, pp. 4171–4186.

[4] GeeksforGeeks. *What is Low Rank Adaptation (LoRA)?* 2024. URL: https://www.geeksforgeeks.org/what-is-low-rank-adaptation-lora/ (visited on 12/14/2024).

Appendix: AI Usage

AI tools (Claude and ChatGPT) were used throughout this project for the following purposes:

- **Code structuring:** Formatting Python code for readability, adding comments, and improving visual organization
- **Writing assistance:** Rephrasing sentences for clarity and coherence, fixing LaTeX syntax errors
- **Conceptual clarification:** Understanding unclear parts of the original LoRA paper [1] and related mathematical concepts

All core implementation, experimental design, and analysis were conducted independently by the team. AI tools served as assistants rather than primary authors, with all final decisions made by team members.